Algorithm Writing Guidelines :

Algorithms are language independent procedures for solving problems, unlike programs which require specific syntax.

Key Differences from program :

- No data types specified (decided later during programming) .
- No variable declarations needed (direct use allowed).
- Focus on logic, not language specific details.

Flexible syntax options :

You can use any clear notation that's understandable to your team :

Symbol	Example
C-like	`{ }` for blocks, `=` for assignment
Pascal-like	`BEGIN . . . END` , `:=` for assignment
Arrow	← or → for assignment

Goal :

Write so you and your project team can understand it.

Algorithm Analysis Criteria :

Analyze algorithm on efficiency metrics to ensure they solve problems optimally.

Primary Criteria :

1. **Time Efficiency** : How fast does it run ? (Measured as time function).
2. **Space Efficiency** : How much memory does it consume ?

Second Criteria :

- Data transfer (network / internet apps)

- Power Consumption (mobile / handheld devices)
- CPU registers (device drivers / system programming)
- Other based on project requirements.

Time Analysis Method :

Each simple statement takes 1 unit of time (basic level analysis).

Example :

```
ALGORITHM sum(a, b)
  s ← a + b
  RETURN s
```

- 3 Statements → Time function : f(n) = 3 (constant)

Key Assumption :

- Ignore statement complexity (e.g., `x ← a + 6*b` still = 1 unit).
- Nested / procedure calls need separate analysis.
- Results in constant time for simple cases not dependent on input size n.

Analysis Level :

- **Basic** : 1 unit per statement.
- **Detailed** : count machine code instructions like rocket mission planning vs car trip.

Space Analysis Method :

Each variable takes 1 word of space abstract unit.

Example : Sum algorithm space

- **Variables** : a, b (parameters), s (local) → 3 variables = 3 words.
- **Space functions** : constant (order 1)

Note :

- Don’t specify bytes (int / float unknown until programming).
- Constants like 3 or 3000 both represented as O(1).